

# Job Hunt AI

A five-agent pipeline that scrapes fresh postings, scores them against your real background, and writes tailored application materials — then **stops, and hands the final click to a human.**

|             |                |                    |                    |
|-------------|----------------|--------------------|--------------------|
| 5<br>AGENTS | 2.5×<br>FASTER | 4<br>LLM PROVIDERS | \$0<br>COST TO RUN |
|-------------|----------------|--------------------|--------------------|

THE PROBLEM

## The slow part isn't clicking Apply

It's finding fresh relevant postings every day, working out which ones actually fit, and rewriting a resume and cover letter for each one. That's hours a day of research and writing — the part that doesn't scale, and the part that burns people out before they get a reply. Job Hunt AI automates exactly that, and nothing past it.

ARCHITECTURE

## Five agents on a LangGraph state machine

Each stage is a node that reads what it needs from one shared pipeline state and writes back its results. Order matters here — every stage depends on the one before it.

01

### SCOUT

Parses the resume PDF into structured data with one cached LLM call, then scrapes fresh postings across LinkedIn, Indeed and Naukri — deduplicated across sources. Naukri runs on self-healing selectors that survive site markup changes.

jobspy   Scrapling   → resume · jobs[]

02

### MATCHER

Three stages, cheapest first. Local FAISS embeddings strip out the obvious noise for free, a cross-encoder reranker narrows that shortlist, and only then does a paid LLM read each posting against the resume and score fit 0–100 — catching the hard requirements embeddings miss, like "8+ years required".

FAISS · MiniLM   Nemotron rerank   concurrent   → matches[]

03

### TAILOR

For each match above threshold: a rewritten summary, re-ordered resume bullets that mirror the posting's language, the ATS keywords the resume is missing, and a full cover letter. Hard-constrained by prompt to only re-word what's genuinely on the resume — it cannot invent experience.

≤ 8 kits / run   concurrent   → kits[]

04

## REVIEW CREW

A skeptical hiring-manager persona critiques every cover letter — generic filler, unbacked claims, anything that reads as machine-written — then an editor persona rewrites it against that critique, holding every factual claim fixed.

critic → editor

concurrent

→ polished letters

05

## APPLIER

Writes each kit to readable Markdown, records it in a SQLite tracker with dedupe, and notifies you. Applications then move through a real funnel — prepared, applied, interviewing, offer — in the dashboard.

SQLite

Slack · email

→ applied[]

### PERFORMANCE

# 2.5× faster by bounding concurrency

Matcher fit-scoring, Tailor kit generation and the Review exchange each run their LLM calls concurrently rather than one at a time — capped by a configurable worker pool so the pipeline speeds up without tripping provider rate limits.

| STAGE                    | CALLS / RUN | SEQUENTIAL | CONCURRENT ×4 | SPEEDUP |
|--------------------------|-------------|------------|---------------|---------|
| Matcher — fit scoring    | 8           | ~31s       | ~18s          | 1.7×    |
| Tailor — kit generation  | 8           | ~4.6 min   | ~1.8 min      | 2.5×    |
| Review — critique + edit | 16          | ~54s       | ~19s          | 2.9×    |
| All LLM-bound stages     | —           | ~6.0 min   | ~2.4 min      | ~2.5×   |

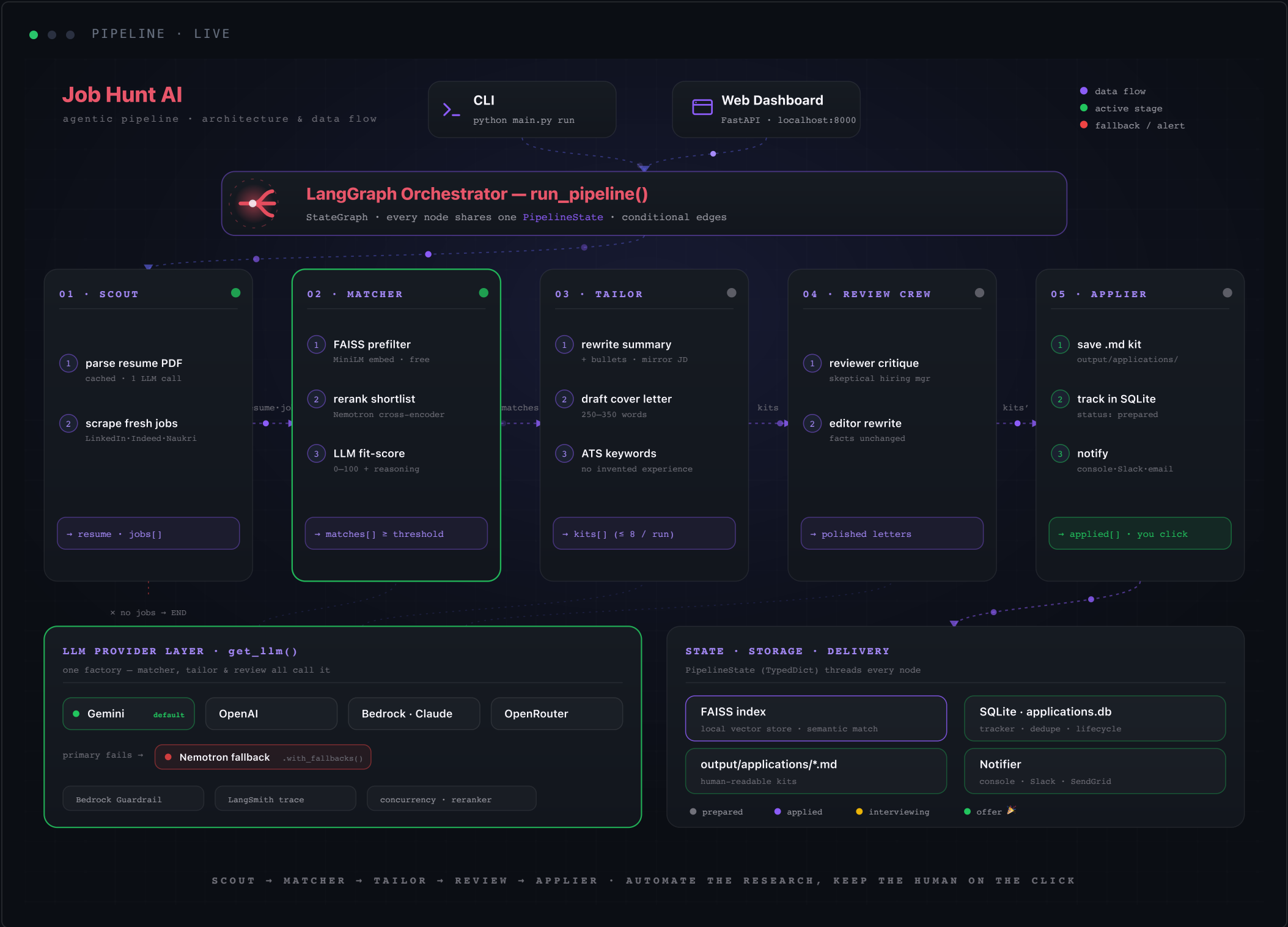
Measured live against a free-tier provider at n=4 per stage, extrapolated to production call counts. The Review row is estimated from Matcher's measured per-call latency rather than benchmarked directly — a critique call is a similar shape to a fit-score call.

**Two changes that look like performance work but aren't.** The reranker is a *cost* optimisation — it roughly halves how many jobs reach the paid scoring step, and its own call costs about nine seconds. The provider fallback is a *reliability* feature — zero overhead on the happy path, and it only engages when the primary provider errors or rate-limits. One saves money; the other turns a crashed run into a slower one that finishes.

### SYSTEM DESIGN

# Data flow, end to end

Clients feed one orchestrator; the orchestrator threads a single typed state through every agent; all three generation stages call one shared LLM factory, so switching providers is a config change rather than a refactor.



ENGINEERING DECISIONS

# The tradeoffs behind the build

Most of the interesting work in this project wasn't wiring agents together — it was deciding what each layer should and shouldn't be responsible for.

## Why three matching stages instead of just asking a model?

Embeddings are cheap and good at topic similarity, but blind to hard requirements. LLMs catch those but cost money per job. So the cheap filter runs first on everything, a free reranker narrows what survives, and the paid model only ever sees a shortlist that already looks plausible. **Right tool, right position in the funnel.**

## Why a reranker if it doesn't make anything faster?

Because it roughly halves how many jobs reach the expensive scoring step. It runs on a separate free budget, so it doesn't consume the same rate limit the generation stages need. **It buys cost headroom, not latency** — and conflating those two would have led to optimising the wrong thing.

## Why fallbacks rather than retries?

A retry against a provider that just rate-limited you usually fails again. Every agent calls one shared factory, so wrapping that single function in a fallback means any provider failure transparently reroutes to a free model mid-run. **One place to change, whole pipeline gets resilient.**

## Why cap concurrency at four?

Unbounded parallelism is the fastest way to get rate-limited into failure. A bounded worker pool captures most of the speedup while staying inside free-tier limits, and it's a single config value for anyone whose provider allows more. **Fast enough, and still polite.**

Why not auto-submit  
the applications?

Auto-filling forms on job platforms violates their terms and gets accounts banned — the worst possible outcome mid-search — and recruiters increasingly filter obviously automated applications. **Automate the research and writing. Keep a human on the part that carries real risk.**

STACK

Built to run anywhere

Local on a free tier, in Docker, or serverless on AWS — the same pipeline, switched by configuration rather than forked code.

|                                                                                           |                                                                                      |                                                                                    |                                                                          |                                                                                  |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <div>ORCHESTRATION</div> <div>LangGraph<br/>LangChain<br/>Python 3.10+</div>              | <div>RETRIEVAL</div> <div>FAISS<br/>MiniLM embeddings<br/>Cross-encoder rerank</div> | <div>MODELS</div> <div>Gemini<br/>OpenAI<br/>Bedrock · Claude<br/>OpenRouter</div> | <div>INTERFACE</div> <div>FastAPI<br/>Vanilla JS<br/>No build step</div> | <div>DATA</div> <div>SQLite tracker<br/>Markdown kits<br/>Slack · SendGrid</div> |
| <div>DELIVERY</div> <div>Docker Compose<br/>GitHub Actions CI<br/>Bedrock AgentCore</div> |                                                                                      |                                                                                    |                                                                          |                                                                                  |

Automate the research and the writing.  
Keep the human on the click.